

cualautocompro

Guia de despliegue en hosting cPanel

Angular 22 + Node.js/Express + Prisma + MariaDB 10.5+

Stack del proyecto

- apps/frontend - Aplicacion Angular 22 (SPA)
- apps/backend - API REST en Node.js + Express + Prisma
- Base de datos - MariaDB 10.5+ (charset utf8mb4 obligatorio)

Esta guía describe paso a paso cómo desplegar el proyecto completo en un hosting con cPanel. Reemplazados por String en constantes validadas sobre Node.js y MariaDB con utf8mb4 obligatorio.

Indice

1. Vision general y requisitos
2. Consideraciones criticas antes de empezar
3. Preparacion local del proyecto
4. Configuracion de la base de datos MariaDB
5. Despliegue del backend Node.js en cPanel
6. Despliegue del frontend Angular
7. Configuracion de SSL y dominio
8. Verificacion final y monitoreo
9. Problemas frecuentes
10. Alternativas recomendadas
11. Login social con Google y Apple (OAuth 2.0 / OIDC)

1. Vision general y requisitos

El proyecto **cualautocompro** es un monorepo con dos aplicaciones: un frontend Angular 22 (SPA) que se sirve como archivos estaticos, y un backend Node.js/Express que expone una API REST con Prisma sobre **MariaDB 10.5+** (provider = 'mysql' en el schema).

Arquitectura objetivo en cPanel

Visitante Navegador	Frontend Angular Archivos estaticos Apache en public_html	Backend Node.js API REST (Express) Puerto interno	MariaDB 10.5+ utf8mb4 obligatorio Prisma ORM
-------------------------------	--	--	---

Requisitos del lado del cliente

- bullet Computadora con Node.js 20+ y npm 10+ (incluido con Node 20).
- bullet Acceso al panel cPanel (URL, usuario y contraseña).
- bullet Cliente MariaDB local o acceso a phpMyAdmin para preparar la BD.
- bullet Cliente FTP o acceso al Administrador de archivos de cPanel.
- bullet Dominio configurado apuntando a los DNS del hosting.

Requisitos del lado del hosting

- bullet Plan cPanel con soporte para aplicaciones Node.js ('Setup Node.js App').
- bullet MariaDB 10.5+ o MySQL 8+ (MariaDB es lo habitual en cPanel compartido).
- bullet Acceso SSH (recomendado) o Terminal desde el Administrador de archivos.
- bullet Al menos 1 GB de espacio libre para node_modules y builds.

2. Consideraciones críticas antes de empezar

MariaDB 10.5+ es lo habitual en cPanel y es lo que usa este proyecto. No use MySQL 5.7 (no soporta JSON nativo con la sintaxis que Prisma necesita). Verifique con su proveedor la versión exacta del servidor MariaDB.

Limitaciones habituales en cPanel compartido

- bullet** **Node.js:** solo en planes con 'Setup Node.js App' o 'Node.js Selector'. En compartido suele estar limitado a 1-2 aplicaciones.
- bullet** **MariaDB remoto:** la mayoría de proveedores no expone el puerto 3306 hacia afuera; el backend debe correr en el mismo servidor.
- bullet** **Procesos persistentes:** el backend Node.js se gestiona desde 'Application Manager'.
- bullet** **Memoria/CPU:** el build de Angular puede requerir más de lo que ofrece el compartido.
- bullet** **SSL:** use Let's Encrypt disponible de forma gratuita en cPanel.

Especificidades de MariaDB que aplican al proyecto

- bullet** **Charset utf8mb4 obligatorio:** el schema Prisma incluye caracteres Unicode (emojis, tilde, ñeie). El CREATE DATABASE y la URL deben especificar `charset=utf8mb4`.
- bullet** **ENUM no se usa en el schema:** los enums de Prisma (Segment, Fuel, etc.) generan columnas inline ENUM(...) en MariaDB que no se pueden extender en runtime. El proyecto usa **String + constantes** validadas en services (SEGMENTS, FUELS, TRANSMISSIONS).
- bullet** **galleryUrls es columna JSON:** ya no es String[] (que MariaDB no soporta nativamente); se serializa como JSON y se normaliza en services con `toGalleryUrls`.
- bullet** **ANSI_QUOTES:** el proyecto usa backticks en sus raw SQL para compatibilidad con servidores MariaDB configurados con este modo.

Decisiones que debiera tomar

- bullet** **MariaDB local o gestionada:** use el MariaDB del propio cPanel o uno externo.
- bullet** **Subdominio para la API:** por convención `api.midominio.com` apunta al backend.
- bullet** **URLs del frontend:** el frontend debe apuntar a la URL pública del backend.

3. Preparacion local del proyecto

Paso 3.1 - Instalar dependencias

```
npm install
```

Paso 3.2 - Configurar variables de entorno

Copie los archivos de ejemplo:

```
cp apps/backend/.env.example apps/backend/.env  
nano apps/backend/.env # o su editor preferido
```

Variables disponibles (ver tambien Paso 5.5 en la seccion 5):

- bullet DATABASE_URL:** `mysql://USER:PASS@HOST:3306/DB?charset=utf8mb4`.
- bullet JWT_SECRET:** minimo 32 caracteres aleatorios en produccion.
- bullet JWT_EXPIRES_IN:** por defecto `7d`.
- bullet PORT:** puerto interno del backend (cPanel suele asignarlo).
- bullet WEB_ORIGIN:** URL exacta del frontend (usada por CORS y redirects post-login).
- bullet BACKEND_ORIGIN:** URL publica del backend. Mismo valor que `WEB_ORIGIN` si estan en el mismo host. Necesario para OAuth (callback URL que passport pasa a Google/Apple).
- bullet ADMIN_EMAIL y ADMIN_INITIAL_PASSWORD:** credenciales del admin que crea el seed. `ADMIN_INITIAL_PASSWORD` debe ser **distinto de admin1234** en produccion o el backend rechaza arrancar.
- bullet GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET:** opcionales. Solo necesarios si se quiere login con Google.
- bullet APPLE_CLIENT_ID / APPLE_KEY_ID / APPLE_TEAM_ID / APPLE_PRIVATE_KEY:** opcionales. `APPLE_PRIVATE_KEY` recibe el PEM del `.p8` con saltos de linea escapados como `\n` literal. Solo si se quiere login con Apple.
- bullet NODE_ENV:** `production` en el servidor.

Paso 3.3 - Compilar el backend

```
npm run build -w apps/backend  
# Salida: apps/backend/dist/
```

Paso 3.4 - Compilar el frontend Angular

```
cd apps/frontend  
npx ng build --configuration production  
# Salida: apps/frontend/dist/frontend/browser/
```

Paso 3.5 - Pruebas locales finales

```
# Levantar MariaDB local (Docker o brew services start mariadb)
cd apps/backend && npm start

# En otra terminal
cd apps/frontend && npx ng serve
```

No continúe hasta haber verificado que todo funciona localmente con MariaDB.

4. Configuración de la base de datos MariaDB

El proyecto usa MariaDB 10.5+ con Prisma (provider = 'mysql'). El charset utf8mb4 es obligatorio en CREATE DATABASE y en la URL de conexión.

Paso 4.1 - Crear la base de datos en cPanel

- 1 Entre a cPanel y vaya a **MySQL Databases** (o 'Databases' -> 'MySQL').
- 2 Cree una base de datos, por ejemplo *usuario_cualauto* (cPanel suele anteponer el usuario).
- 3 Cree un usuario con contraseña robusta (ej. *usuario_app*).
- 4 Asocie el usuario a la base con privilegios **ALL PRIVILEGES**.
- 5 Anote: nombre de BD, usuario, contraseña y host (habitualmente *localhost*).

En cPanel compartido el host de MariaDB suele ser *localhost* (no la IP del servidor). Si el backend corre en otro host, use la IP o dominio del servidor MariaDB.

Paso 4.2 - Asegurar charset utf8mb4

Verifique desde phpMyAdmin (cPanel -> phpMyAdmin) que la base creada use utf8mb4. Si fue creada con otro charset, ajuste con esta consulta SQL:

```
-- Desde phpMyAdmin o cliente mariadb contra la BD de produccion
ALTER DATABASE usuario_cualauto
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

-- Para cada tabla que Prisma cree:
ALTER TABLE Brand CONVERT TO CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Paso 4.3 - Construir la cadena DATABASE_URL

```
# Formato general
DATABASE_URL=mysql://USUARIO:CONTRASENA@HOST:3306/NOMBRE_BD?charset=utf8mb4

# Ejemplo realista en cPanel compartido
DATABASE_URL=mysql://usuario_app:C14v3.Fu3rt3!@localhost:3306/usuario_cualauto?charset=utf8mb4

# Importante
# 1. charset=utf8mb4 SIEMPRE debe ir al final
# 2. Use 'localhost' y no la IP si todo corre en el mismo servidor
# 3. Escape los caracteres especiales en la contraseña (@ # ! etc.)
```

Paso 4.4 - Sobre el GRANT global para shadow DB

En desarrollo se usa *prisma migrate dev*, que requiere crear una **shadow database** para detectar drift. Esto obliga a dar permisos globales (*.*) al usuario MariaDB. **En producción esto NO es necesario**: usamos *prisma migrate deploy*, que solo aplica migraciones y no crea shadow DB.

Por seguridad, el usuario de produccion solo debe tener privilegios sobre la base del proyecto. El GRANT global solo se usa para el usuario de desarrollo/migrations.

5. Despliegue del backend Node.js en cPanel

Esta sección describe el flujo estándar de deploy del backend en cPanel, **compilando directamente en el server**. A diferencia del flujo que sube *dist/* pre-compilado, este flujo requiere que las *devDependencies* (typescript, tsx, @types/*) estén presentes en *package.json*, porque el build se ejecuta en el server. La mayor parte del setup se hace desde el panel de cPanel (**Setup Node.js App**, gestionado por **Passenger**); los comandos de build y Prisma se corren dentro del **ambiente virtual** que cPanel crea para la app.

Paso 5.1 - Verificar dependencias de desarrollo en package.json

Como el build se hace en el server, las *devDependencies* son obligatorias. Confirme que *apps/backend/package.json* las declara:

```
"devDependencies": {
  "typescript": "~6.0.2",
  "tsx": "4.22.4",
  "@types/node": "20.19.43",
  "@types/express": "4.17.25",
  "@types/bcrypt": "5.0.2",
  "@types/cookie-parser": "1.4.10",
  "@types/jsonwebtoken": "9.0.10",
  "@types/multer": "2.2.0",
  "@types/supertest": "6.0.3",
  "supertest": "7.2.2",
  "vitest": "^4.0.8"
}
```

Por que importan las *devDependencies* aqui: este flujo NO usa *--omit=dev*. Se necesita *typescript* para *npm run build*, *tsx* para ejecutar el seed, y los *@types/** para que TypeScript compile sin errores.

Paso 5.2 - Subir solo lo esencial al server

El backend NO debe ir dentro de *public_html*. Cree una carpeta fuera del alcance del servidor web, normalmente en el home del usuario de cPanel, y suba unicamente lo que se compilara en el server:

```
~/cualauto-backend/
+- src/                (codigo TypeScript, se compila en el server)
+- package.json
+- package-lock.json   (lockfile para npm install reproducible)
+- tsconfig.json        (config TypeScript del backend)
+- tsconfig.base.json   (base, extendida por tsconfig.json)
+- prisma/
|   +- schema.prisma
|   +- migrations/      (incluye 20260703221041_init/)
|   +- seed.ts
+- .env                 (variables de entorno, por canal seguro)
+- .env.example         (plantilla, sin secretos)
```

bullet **Que NO debe subirse:** *node_modules/* (se regenera en el server), *dist/* (se compila en el server), *vendor/* (se genera en el server con *prisma generate*), *apps/backend/public/*, *.angular/cache/*, archivos basura (*.DS_Store*, *Thumbs.db*).

Metodos de subida

- bullet **Git (recomendado si cPanel provee terminal SSH):** clone el repositorio directamente. Así el deploy se reduce a *git pull* y luego los pasos de build.
- bullet **SFTP:** use FileZilla o Cyberduck para subir la carpeta. Excluya *node_modules/*, *dist/*, *vendor/*, *.env* (subir aparte por canal seguro).
- bullet **Administrador de archivos de cPanel:** suba un *.zip* con solo lo esencial y descomprima en el server.

.env esta en .gitignore y debe subirse por separado (SFTP, SSH o el editor del panel). Nunca commitarlo.

Paso 5.3 - Registrar la aplicacion en Passenger (Setup Node.js App)

Vaya a cPanel -> **Setup Node.js App** (o 'Software' -> 'Node.js Selector') y registre la aplicacion:

- 1 Haga clic en **Create Application**.
- 2 Configure los campos:

- bullet Node.js version: **20.x** (es el valor del archivo *.nvmrc*).
- bullet Application mode: **Production**.
- bullet Application root: *cualauto-backend* (la ruta del Paso 5.2).
- bullet Application URL: por ejemplo, *api.midominio.com*.
- bullet Application startup file: *dist/src/index.js* (resultado de *npm run build*; lo crearemos en el Paso 5.5).
- bullet Application port: anote el puerto asignado por cPanel.

Por que *dist/src/index.js* y no *dist/index.js*? El *tsconfig.json* del backend tiene *rootDir: '.'* e *include: ['src', '__tests__']*, por lo que TypeScript preserva la estructura al compilar: el entry point queda en *dist/src/index.js*.

- 1 Tras crear la aplicacion, pulse **'Run NPM Install'** (tambien aparece como **'Ensure dependencies'** en versiones recientes de cPanel). Esta accion delega en **Passenger** la instalacion de dependencias (incluyendo devDependencies), lo cual es util cuando la terminal SSH esta limitada o *npm install* directo falla por LVE/CloudLinux.
- 2 Espere a que termine. Passenger creara *node_modules/* con tanto dependencies como devDependencies.

Que hace 'Ensure dependencies': Passenger ejecuta *npm install* en un proceso con su propio sandbox de memoria, evitando los limites LVE que pueden romper *prisma generate* cuando se corre desde la terminal compartida. Si aun asi falla, ejecute *npx prisma generate* manualmente en el Paso 5.5.

Paso 5.4 - Instalar Node y activar el ambiente virtual

Una vez registrada la aplicación, cPanel crea un **entorno virtual de Node aislado** para esa app (con su propia versión de node y npm, y un PATH independiente). El panel muestra el comando exacto en la sección **'Enter to the virtual environment'** de la app registrada. Típicamente luce así:

```
# Reemplace <usuario> por su usuario de cPanel
source /home/<usuario>/nodeenv/cualauto-backend/20/bin/activate

# Verificar que apunta al node del ambiente aislado:
which node
# /home/<usuario>/nodeenv/cualauto-backend/20/bin/node

node --version
# v20.x.x
```

Por qué activar el ambiente virtual? Sin activarlo, los comandos *node* y *npm* que ejecute pueden ser los del sistema (otra versión, sin las deps de la app), lo que rompe el build o la ejecución de Prisma. Active el ambiente SIEMPRE antes de los comandos del Paso 5.5.

Si la versión de Node 20 no aparece disponible en cPanel, el hosting puede no tenerla. En ese caso contacte a su proveedor o considere las **Alternativas recomendadas** en la sección 10.

Paso 5.5 - Compilar y aplicar esquema (build + Prisma + seed)

Con el ambiente virtual activo y posicionado en la raíz del backend (`cd ~/cualauto-backend`), ejecute en orden:

```
cd ~/cualauto-backend

# 1. Generar el cliente Prisma (necesario antes del build)
npx prisma generate

# 2. Compilar TypeScript -> dist/
npm run build

# 3. Aplicar migraciones pendientes (no crea nuevas)
npx prisma migrate deploy

# 4. Sembrar datos iniciales (marcas, modelos, admin)
npx tsx prisma/seed.ts
```

Que hace cada comando

- bullet** **npx prisma generate:** genera el cliente Prisma en `node_modules/.prisma/client/`. Aunque Passenger ya corrió `npm install` (Paso 5.3), este paso es idempotente y asegura que el cliente este actualizado con `schema.prisma`.
- bullet** **npm run build:** ejecuta `tsc -p tsconfig.json` y produce `dist/src/index.js` (el entry point que Passenger arrancara).
- bullet** **npx prisma migrate deploy:** aplica las migraciones SQL pendientes (la inicial `20260703221041_init/` crea todas las tablas: Brand, Model, Version, EquipmentItem, VersionEquipment, MaintenanceCost, User, Comparison, ComparisonItem, Favorite).
- bullet** **npx tsx prisma/seed.ts:** pobla la BD con datos de ejemplo y crea el usuario admin (`ADMIN_EMAIL` + `ADMIN_INITIAL_PASSWORD` del `.env`).

Orden importante: *prisma generate* antes de *npm run build*, porque TypeScript importa tipos desde *@prisma/client*. El hook *prebuild* del *package.json* ya invoca *prisma generate* automáticamente, pero ejecutarlo explícito es más claro para diagnosticar.

Si *npx prisma generate* falla con *'WebAssembly.Instance(): Out of memory'* (límite LVE de CloudLinux), pre-genera el cliente en una máquina local con RAM suficiente (*npm run db:generate -w apps/backend && npm run vendor:prisma -w apps/backend*) y suba la carpeta *vendor/prisma-client/* junto al resto del código. Passenger la recogerá automáticamente al reiniciar.

Paso 5.6 - Reiniciar la aplicación Node

En cPanel -> **Setup Node.js App**, pulse **Restart** sobre la app *cualauto-backend*. Passenger detendrá el proceso actual y arrancará *node dist/src/index.js* dentro del ambiente virtual.

- bullet Verifique los logs desde el panel o en *~/cualauto-backend/.cpanel_node.yml*.
- bullet Si el restart queda en bucle, revise *DATABASE_URL*, *JWT_SECRET* y que *ADMIN_INITIAL_PASSWORD* no sea *admin1234*.

Tras editar *.env* siempre hay que reiniciar para que Passenger recargue las variables. El reload automático solo aplica a cambios de código, no de entorno.

Paso 5.7 - Verificar el backend

```
# Health check (el endpoint /health existe en este proyecto)
curl https://api.midominio.com/health
# Respuesta esperada:
# {"data":{"status":"ok","env":"https://midominio.com"}}
```

El endpoint */health* confirma que el backend arranca, que la variable *WEB_ORIGIN* se cargó correctamente y que la conexión a MariaDB funciona. Si responde OK, el deploy básico está completo. Para troubleshooting, ver sección 9.

Si prefiere verificar dentro del ambiente virtual antes de exponer el backend, ejecute: *curl http://localhost:PUERTO_ASIGNADO/health* tras hacer Restart. Passenger expone la app en localhost antes del proxy reverso.

6. Despliegue del frontend Angular

Paso 6.1 - Configurar la URL del backend en environments

El frontend usa la convencion estandar de Angular con dos archivos de environment y *fileReplacements* en *angular.json*. En build de desarrollo se usa *environment.ts* (apunta a *localhost:3000*); en build de produccion se reemplaza por *environment.prod.ts* con la URL del backend desplegado.

Estructura de archivos

```
apps/frontend/
+- src/
|   +- environments/
|   |   +- environment.ts           (dev: localhost:3000)
|   |   +- environment.prod.ts     (prod: URL real)
|   +- app/
|   |   +- core/
|   |   |   +- env.ts               (punto de acceso, importa de environments/environment)
+- angular.json                     (define fileReplacements)
```

Paso 6.1.1 - Verificar/crear environment.ts (dev)

```
// apps/frontend/src/environments/environment.ts
export const environment = {
  production: false,
  apiBase: 'http://localhost:3000/api/v1',
} as const;
```

Este archivo ya existe en el proyecto. Si necesitas crearlo, hazlo en ***apps/frontend/src/environments/environment.ts***. La clave exportada debe llamarse **environment** (Angular CLI busca este nombre por convencion) y debe incluir **apiBase** con la URL completa del backend (incluyendo el path **/api/v1**).

Paso 6.1.2 - Configurar environment.prod.ts (produccion)

```
// apps/frontend/src/environments/environment.prod.ts
export const environment = {
  production: true,
  apiBase: 'https://api.midominio.com/api/v1',
} as const;
```

IMPORTANTE: la URL DEBE terminar en **/api/v1**. El backend monta todas las rutas bajo ese prefijo (ver *apps/backend/src/app.ts*: *app.use('/api/v1/...')*). Si omites **/api/v1**, el frontend llamara a **https://api.midominio.com/models** y el backend respondera 404 porque la ruta real es **https://api.midominio.com/api/v1/models**.

Reemplaza **api.midominio.com** por el dominio real del backend (el mismo que configuraste en **Application URL** del Setup Node.js App de cPanel). Esta URL debe coincidir con **WEB_ORIGIN** en el **.env** del backend, porque el backend la usa en la validacion CORS.

Paso 6.1.3 - Verificar fileReplacements en angular.json

El bloque `configurations.production` de `apps/frontend/angular.json` debe incluir un `fileReplacements` que apunte a `environment.prod.ts`:

```
// apps/frontend/angular.json (fragmento)
"configurations": {
  "production": {
    "budgets": [ ... ],
    "outputHashing": "all",
    "fileReplacements": [
      {
        "replace": "src/environments/environment.ts",
        "with": "src/environments/environment.prod.ts"
      }
    ]
  },
  "development": { ... }
}
```

Esta configuración hace que al ejecutar `ng build --configuration production`, Angular CLI reemplace automáticamente `environment.ts` por `environment.prod.ts` en el bundle final. No es necesario editar el código fuente en cada build; solo cambia el contenido de `environment.prod.ts` antes de cada deploy.

Paso 6.1.4 - Verificar env.ts (punto de acceso)

El archivo `apps/frontend/src/app/core/env.ts` es el único punto que lee `environment`. Lo importa y expone la constante `ENV` que usa el resto del código (`ApiService`, etc.).

```
// apps/frontend/src/app/core/env.ts
import { environment } from '../../../environments/environment';

export const ENV = {
  apiBase:
    (typeof window !== 'undefined' &&
      (window as { __env?: { apiBase?: string } }).__env?.apiBase) ||
    environment.apiBase,
  production: environment.production,
} as const;
```

El fallback a `window.__env` se mantiene por compatibilidad con despliegues que prefieran inyectar la URL en runtime via un `<script>` en `index.html`. En producción normal, `environment.prod.ts` es el que se usa (gracias al `fileReplacement`) y `window.__env` no existe.

Paso 6.1.5 - Compilar frontend para producción

```
cd apps/frontend

# Build + verificación automática del bundle:
npm run build:prod
# Equivale a: ng build --configuration production && node scripts/verify-frontend-bundle.cjs

# Salida esperada:
# [verify] apiBase en el bundle: https://api.midominio.com/api/v1
# [verify] OK: apiBase no apunta a localhost
# [verify] OK: apiBase usa https
# [verify] Prefijo detectado del backend: /api/v1
# [verify] OK: apiBase termina en /api/v1 (correcto, coincide con el backend)
# [verify] TODAS LAS VERIFICACIONES PASARON

# Si la verificación FALLA, el build NO es válido para producción.
# Corrige environment.prod.ts y vuelve a correr build:prod.
```

El script *verify-frontend-bundle.cjs* lee el código fuente del backend (*apps/backend/src/app.ts*) y detecta automáticamente el prefijo que usa el backend (ej: */api/v1*). Luego valida que el *apiBase* del frontend termine exactamente en ese prefijo. Esto elimina la posibilidad de olvidar el sufijo (causa más común de 404 en producción) aunque el prefijo del backend cambie en el futuro.

Que verifica concretamente: 1. Que el bundle existe en *dist/frontend/browser/* 2. Que *apiBase* no apunta a localhost (no es un build de dev) 3. Que *apiBase* usa https 4. Que *apiBase* termina en el prefijo detectado del backend 5. Que hay exactamente un valor de *apiBase* en el bundle Si alguna falla, el script sale con código 1 y el deploy no debe continuar. Puedes correrlo solo con: *npm run verify:bundle*

Paso 6.2 - Subir los archivos estáticos a *public_html*

- 1 Localice la carpeta *apps/frontend/dist/frontend/browser/*.
- 2 En cPanel abra el Administrador de archivos y vaya a *public_html*.
- 3 Suba el contenido de *browser/* dentro de *public_html*.
- 4 Verifique que *index.html* quede en la raíz.

Paso 6.3 - Configurar reescritura de URL (*.htaccess*)

```
# public_html/.htaccess
<IfModule mod_rewrite.c>
  RewriteEngine On
  RewriteBase /
  RewriteRule ^index\.html$ - [L]
  RewriteCond %{REQUEST_FILENAME} !-f
  RewriteCond %{REQUEST_FILENAME} !-d
  RewriteRule . /index.html [L]
</IfModule>

<IfModule mod_deflate.c>
  AddOutputFilterByType DEFLATE text/html text/css application/javascript application/json
</IfModule>

<IfModule mod_expires.c>
  ExpiresActive On
  ExpiresByType text/css "access plus 1 month"
  ExpiresByType application/javascript "access plus 1 month"
  ExpiresByType image/png "access plus 6 months"
</IfModule>
```

Paso 6.4 - Configurar WEB_ORIGIN en el backend (CORS)

El backend valida CORS comparando el header *Origin* con **WEB_ORIGIN**. El match debe ser **exacto** (esquema + host + puerto si no es 443):

```
# apps/backend/.env
WEB_ORIGIN=https://midominio.com

# Si sirve el frontend en www.midominio.com y no en midominio.com,
# ajuste WEB_ORIGIN a esa URL exacta. No admite multiples origenes.
```

Si el navegador reporta 'CORS policy: No Access-Control-Allow-Origin', revise que el valor de **WEB_ORIGIN** coincida exactamente con el origen que envía el navegador.

Paso 6.5 - Verificar el frontend

- 1 Abra <https://midominio.com> en el navegador.
- 2 Verifique que la pagina principal carga sin errores (F12 -> Console).
- 3 Navegue a una ruta interna (ej. [/catalogo](#)) y verifique que NO aparece 404.
- 4 Realice una peticion al backend: deberia obtener respuesta JSON sin error CORS.

7. Configuración de SSL y dominio

Paso 7.1 - SSL con Let's Encrypt

- 1 En cPanel vaya a **SSL/TLS Status** o **Let's Encrypt**.
- 2 Seleccione el dominio principal y los subdominios (ej. *api.midominio.com*).
- 3 Haga clic en **Issue** y espere unos segundos.
- 4 Active **Force HTTPS Redirect**.

Paso 7.2 - DNS si su dominio esta en otro proveedor

- bullet Cambie los nameservers al hosting, o cree registros A apuntando a la IP del servidor.
- bullet Subdominio *api.midominio.com*: registro A -> IP del hosting.
- bullet Verifique con *nslookup midominio.com* o *dig midominio.com*.

Paso 7.3 - Cabeceras de seguridad

```
# Anada al .htaccess del frontend
<IfModule mod_headers.c>
  Header set Strict-Transport-Security "max-age=31536000; includeSubDomains" env=HTTPS
  Header set X-Content-Type-Options "nosniff"
  Header set X-Frame-Options "SAMEORIGIN"
  Header set Referrer-Policy "strict-origin-when-cross-origin"
</IfModule>
```

8. Verificación final y monitoreo

Checklist de verificación

Item	Verificación
MariaDB	BD creada con charset utf8mb4; collation utf8mb4_unicode_ci
DATABASE_URL	Termina en ?charset=utf8mb4
Backend /health	GET https://api.midominio.com/health responde JSON ok
Frontend	Carga la página principal sin errores en consola
Rutas SPA	Las rutas internas (/catalogo, etc.) cargan sin 404
CORS	Peticion desde el frontend NO falla por CORS
Login/Auth	Flujo completo de autenticación funciona
Prisma	Consultas a la BD funcionan sin errores de driver
Seed admin	Usuario admin creado y puede iniciar sesión
SSL	Certificado válido; HTTP redirige a HTTPS
Logs	Sin errores críticos en logs del backend

Monitoreo básico

- bullet Active monitoreo de uptime externo (UptimeRobot, Better Uptime, etc.).
- bullet Revise periódicamente los logs en `~/cualauto-backend/*.log` y la sección 'Errors' de cPanel.
- bullet Configure backups automáticos de MariaDB desde cPanel -> Backup.
- bullet Documente cualquier cambio en el servidor en un runbook para el equipo.

9. Problemas frecuentes

Error 'Unknown character set: utf8mb4'

Su servidor MariaDB es muy antiguo. MariaDB 10.5+ soporta utf8mb4 nativamente. Considere actualizar o usar utf8 (sin mb4) aunque pierda soporte de emojis.

Prisma: 'P1001: Can't reach database server'

Verifique DATABASE_URL, que el servicio MariaDB este activo y que el usuario pueda conectarse desde el host del backend.

Prisma: 'P2002: Unique constraint failed' al ejecutar el seed

El seed re-crea versiones y joins con create directo. Si ya hay datos, primero limpie las tablas: `DELETE FROM VersionEquipment; DELETE FROM MaintenanceCost; DELETE FROM Version; DELETE FROM Model;`

Prisma migrate deploy falla con 'Migration not found'

Asegurese de haber subido la carpeta `prisma/migrations/20260703221041_init/` con su archivo `migration.sql`.

npm tsx prisma/seed.ts falla con 'Cannot find module src/config/env.js'

El seed es auto-contenido y carga el `.env` directamente con `dotenv` (no importa `src/config/env`). Si tu seed es de una version anterior a este fix, reemplaza `'import { env } from "../src/config/env.js"'` por `dotenv + zod inline`. Ver Paso 5.5 para los detalles.

El backend no arranca: 'ADMIN_INITIAL_PASSWORD must be overridden'

Cambio la variable en `.env`. El default `'admin1234'` esta bloqueado en produccion por una salvaguarda explicita del proyecto.

Todas las llamadas del frontend al backend dan 404 (ej: GET /models 404)

El apiBase del frontend no incluye el sufijo `/api/v1`. El backend monta todas las rutas bajo `/api/v1`, asi que el frontend debe llamar a `https://api.midominio.com/api/v1/models`, no a `https://api.midominio.com/models`. Edita `src/environments/environment.prod.ts` y agrega `'api/v1'` al final de `apiBase`. Luego regenera el bundle (`npm run build:prod` incluye una verificacion automatica que detecta este error).

El bundle del frontend apunta a localhost despues de hacer build:prod

El `fileReplacement` en `angular.json` no esta aplicando. Verifica que `configurations.production` tenga `'fileReplacements'` apuntando a `src/environments/environment.prod.ts`. Si no esta, agregalo (ver paso 6.1.3).

El backend no arranca: 'JWT_SECRET too short'

`JWT_SECRET` debe tener al menos 16 caracteres. Genere uno con: `node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"`.

CORS: 'No Access-Control-Allow-Origin header'

`WEB_ORIGIN` debe coincidir EXACTAMENTE con el origen que envia el navegador (esquema + host + puerto si no es 443).

Frontend muestra 404 al refrescar rutas internas

Falta el `.htaccess` del paso 6.3 con la reescritura a `index.html`.

Build de Angular muy pesado / se queda sin memoria

Habilite code splitting con `loadChildren` o use `@defer`. En cPanel compartido puede ser necesario un VPS para el build inicial.

Caracteres raros (Ã±, Ã©, etc.) en datos

La BD o las tablas no estan en `utf8mb4`. Aplique `ALTER DATABASE / ALTER TABLE` del paso 4.2.

La aplicacion se cae sola despues de un tiempo

Planes compartidos reciclan procesos. Considere migrar a VPS o plataforma gestionada.

prisma generate falla con 'WebAssembly.Instance(): Out of memory' (limite LVE de CloudLinux)

En el nuevo flujo de deploy, *prisma generate* se corre en el server en el Paso 5.5. Si el LVE de CloudLinux rechaza el WebAssembly de Prisma, la primera linea de defensa es **'Ensure dependencies'** del Paso 5.3 (Passenger corre *npm install* en un sandbox con mas memoria). Si aun asi falla, pre-genere el cliente en una maquina local con RAM suficiente (*npm run db:generate -w apps/backend && npm run vendor:prisma -w apps/backend*), suba la carpeta *vendor/prisma-client/* junto al resto del codigo y luego ejecute *npm run prisma generate* en el Paso 5.5 (Passenger la recogerá al reiniciar).

El backend arranca pero Prisma no conecta: 'P1001 Can't reach database server'

Verifique que `DATABASE_URL` en `.env` apunta al host:puerto correcto, que el usuario MySQL existe, que la base de datos esta creada y que el charset es `utf8mb4`. Tambien confirme que el servidor de BD escucha en la IP que `DATABASE_URL` indica (en MariaDB local suele ser `localhost` o `127.0.0.1`; si el backend esta en otro container/host, use la IP correspondiente).

Los botones OAuth no aparecen despues del deploy

El endpoint `GET /auth/providers` debe devolver `{"data":{"google":true,"apple":false},"error":null}`. Si devuelve `google:false`, las envs `GOOGLE_CLIENT_ID/SECRET` no se cargaron: verifique que las puso en Setup Node.js App -> Environment (no en `.env` manualmente) y que reinicio la app. Si devuelve 404, falta la migracion `20260709120000_oauth_identity`; corra *npm run prisma migrate deploy* en el server. Si devuelve 500, hay un error en el codigo OAuth - revisar logs.

Google muestra 'redirect_uri_mismatch' despues del deploy

La URL registrada en Google Cloud Console no coincide con lo que passport envia. El backend envia `BACKEND_ORIGIN/api/v1/auth/google/callback`. Verifique que la entrada registrada en Google sea exactamente `https://cualautocompro.cl/api/v1/auth/google/callback` (sin `www`, sin `:443`, sin trailing slash, con el path completo). Solo una entrada por environment.

Tras login OAuth el navegador queda en blanco o no aparece logueado

Tres causas probables: (1) el callback devuelve 404 (cPanel proxy bloquea el path `/api/v1/auth/google/callback` - revisar `.htaccess` o `AllowOverride`), (2) `WEB_ORIGIN` en el backend no coincide con el origen real del frontend (verificar `https://`, sin `www`), (3) `Secure: true` en la cookie pero el site se sirve por HTTP. Si todo parece correcto, abrir DevTools -> Network, hacer click en el boton, y revisar que la respuesta Set-Cookie tenga `Secure` y el dominio correcto.

El flujo OAuth funciona en local pero falla en cPanel

Lo mas probable es diferencia entre `WEB_ORIGIN/BACKEND_ORIGIN` en `.env` del server vs el dev. En cPanel ambos suelen ser `https://cualautocompro.cl`. Si el backend esta en un subdominio (ej `api.cualautocompro.cl`), `BACKEND_ORIGIN` debe apuntar ahi. Google y Apple rechazan requests donde el `redirect_uri` de la config del provider no coincida exactamente con el `BACKEND_ORIGIN` + path. Verificar que los redirect URIs en Google Cloud Console y Apple Developer coincidan.

Errores 'invalid_client' o 'JWT signature' con Apple

El `APPLE_PRIVATE_KEY` esta mal formada. La causa #1: el `.p8` original se pego con saltos de linea reales; deben ser `\n` literales. La causa #2: el header `-----BEGIN PRIVATE KEY-----` o footer fueron truncados al pegar. La causa #3: el Team ID o Key ID estan al reves o mal copiados (son 10 caracteres exactos). Regenerar el string con el comando de la seccion 11.4.4.

10. Alternativas recomendadas

Si su plan cPanel no soporta Node.js persistente o MariaDB en la version requerida, considere migrar a una plataforma mas adecuada para el stack. Las opciones mas usadas:

Plataforma	Frontend	Backend	MariaDB	Costo aprox.
Render	Static Site	Web Service Node	Postgres (no MariaDB)	Plan free + ~\$7/mes
Railway	Static Site	Web Service Node	MySQL/MariaDB disponible	\$5/mes + uso
Vercel + Railway	Vercel (Angular)	Railway (Node)	Railway MariaDB	Free + ~\$5/mes
DigitalOcean App Platform	Static	Node	MySQL gestionado	~\$12/mes
VPS (Hetzner, Contabo, etc)	Manual	Manual	MariaDB manual	\$4-\$20/mes
Hostinger Cloud	Si soporta Node	Si soporta Node	MariaDB	\$3-\$10/mes

Recomendacion

Para el stack cualautocompro (Angular + Node + Prisma + MariaDB) la opcion mas simple y confiable es Vercel (frontend Angular) + Railway (backend y BD MariaDB), con despliegues automaticos desde Git. Si requiere mantener cPanel por motivos del cliente, asegurese de que el plan soporte Node.js persistente y MariaDB 10.5+.

11. Login social con Google y Apple (OAuth 2.0 / OIDC)

El sistema soporta inicio de sesión con cuentas de Google y Apple además del tradicional email + contraseña. Esta sección documenta los pasos para activar ambos proveedores en producción. Si solo quiere uno de los dos (por ejemplo Google ahora y Apple después), es perfectamente válido: configure solo las variables y el panel del provider correspondiente; el otro botón simplemente no aparecerá.

Que se necesita configurar

- bullet Backend:** 6 variables de entorno nuevas (Google: 2, Apple: 4).
- bullet Panel del provider:** redirect URI autorizado en Google Cloud Console o Apple Developer.
- bullet Base de datos:** nueva migración Prisma *oauth_identity* que crea la tabla *UserIdentity* y hace *User.passwordHash* opcional.
- bullet Frontend:** sin cambios (ya viene integrado en el build).

11.1 Arquitectura del flujo OAuth

El flujo sigue el patrón estándar OAuth 2.0 Authorization Code con PKCE, implementado con **Passport.js** en el backend. No requiere ningún servicio externo de pago (Auth0, Clerk, Supabase) - todo el control queda en el código del proyecto.

```
[Usuario] -> [Frontend /login]
| click 'Continuar con Google'
v
[Backend GET /auth/google]
| genera state firmado (HS256, 10min TTL) + nonce OIDC
| cookie httpOnly 'oauth_state' en /api/v1/auth
v
[Google accounts.google.com]
| usuario autoriza con su Gmail
v
[Backend GET /auth/google/callback?code=&state=]
| verifica state vs cookie (CSRF protection)
| intercambia code por tokens via POST a oauth2.googleapis.com/token
| valida id_token (JWKS, issuer, aud, exp, email_verified)
| llama OAuthService.resolveUser():
|   1. match por (provider, sub) en UserIdentity -> reutiliza User
|   2. match por email en User -> vincula UserIdentity
|   3. crea User + UserIdentity nuevo
| setea cookie 'auth' (JWT firmado, 7d, secure: true)
v
[Redirect 302 a WEB_ORIGIN + returnUrl?oauth=ok]
| ej: https://cualautocompro.cl/?oauth=ok
v
[Frontend /]
| AuthService.bootstrap() llama /auth/me con la cookie
| currentUser signal queda poblado, header muestra el email del usuario
v
[Logueado. Sin contraseña. Sin base de datos de terceros.]
```

Sobre WEB_ORIGIN vs BACKEND_ORIGIN: ahora hay dos env vars que suelen valer lo mismo en produccion pero tienen propósitos diferentes:

- bullet** **WEB_ORIGIN:** URL publica del FRONTEND. Usada por CORS y para redirigir al usuario después del login.
- bullet** **BACKEND_ORIGIN:** URL publica del BACKEND. Usada por passport como *redirect_uri* que pasa a Google/Apple. Si backend y frontend son el mismo host (caso cPanel), vale lo mismo que *WEB_ORIGIN*.
- bullet** En cPanel con virtual host compartido, ambos son *https://cualautocompro.cl*.

11.2 Aplicar la migración Prisma oauth_identity

Antes del primer deploy con OAuth, aplique la migración en la BD de producción. Sin esto, GET /auth/providers devuelve 500 porque la tabla *UserIdentity* no existe.

Si deploya en el server con el flujo del Capítulo 5.5, el comando se corre automáticamente:

```
cd cualauto-backend # o donde este el backend en el server
source /home/USUARIO/nodeenv/cualauto-backend/20/bin/activate
npx prisma migrate deploy

# Esperado:
# 1 migration found in prisma/migrations
# Applying migration `20260709120000_oauth_identity`
# All migrations have been successfully applied.
```

Idempotente: si ya se aplico, no hace nada. NO usar *prisma migrate dev* en producción (pide shadow DB y hace cambios no revisados).

11.3 Configurar Google Cloud Console

Google requiere registrar el redirect URI exacto que el backend pasara al provider. El backend usa *BACKEND_ORIGIN/api/v1/auth/google/callback*.

- 1 Ir a <https://console.cloud.google.com/apis/credentials>.
- 2 Si nunca configuro pantalla de consentimiento, ir primero a *OAuth consent screen*: User type = External, support email = su email, developer contact = su email. En *Audience* agregar su email como *Test user* mientras la app este en modo Testing.
- 3 Volver a *Credentials* -> **Create credentials** -> **OAuth client ID**.
- 4 Application type: **Web application**.
- 5 Name: ej. *cualautocompro prod*.
- 6 Authorized JavaScript origins: *https://cualautocompro.cl*.
- 7 Authorized redirect URIs: *https://cualautocompro.cl/api/v1/auth/google/callback* (este es **BACKEND_ORIGIN + /api/v1/auth/google/callback**). NO usar :443 ni :3000 ni incluir *www*.
- 8 Create. Anotar **Client ID** y **Client secret**.

Setear en cPanel (Setup Node.js App -> Environment variables):


```
GOOGLE_CLIENT_ID=236383241129-qujrifc5otfpiattfehhsml0t2vm15pe.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSPX-y3aUkziI5aeQoi293t0pkttgHgZ

# Reiniciar la app desde Setup Node.js App para que cargue las envs.
```

NO commitear estos valores en git. El archivo `apps/backend/.env` (y `.env.development`, `.env.local`) están en `.gitignore`. Si el valor se filtra (ej. push accidental), rotar el secret inmediatamente desde Google Cloud Console.

11.4 Configurar Apple Developer

Apple es mas complicado que Google. Requiere un App ID, un Service ID, y un .p8 private key firmado por Apple. Apple solo acepta HTTPS en redirect URIs, lo cual en produccion es automatico.

11.4.1 Crear App ID

- 1 Ir a <https://developer.apple.com/account/resources/identifiers>.
- 2 Click + -> **App IDs** -> Continue.
- 3 Select a Platform (cualquiera sirve para web, pero escoja iOS si tiene app nativa).
- 4 Description: *cualautocompro web*.
- 5 Bundle ID: *cl.cualautocompro.web* (debe ser unico, formato reverse-DNS).
- 6 Capabilities: marcar **Sign in with Apple**.
- 7 Continue -> Register.

11.4.2 Crear Service ID

- 1 Volver a *Identifiers* -> + -> **Service IDs**.
- 2 Description: *cualautocompro web prod*.
- 3 Identifier: *cl.cualautocompro.web.prod* (otro ID unico).
- 4 Capabilities: **Sign in with Apple** -> Configure.
- 5 Primary App ID: el App ID del paso anterior.
- 6 Web Domain: *cualautocompro.cl* (sin https://, sin path).
- 7 Return URLs: *https://cualautocompro.cl/api/v1/auth/apple/callback*.
- 8 Save -> Continue -> Register.

11.4.3 Crear Private Key (.p8)

- 1 Ir a *Keys* -> +.
- 2 Key Name: *cualautocompro-signin*.
- 3 Sign in with Apple: marcado -> Configure -> Primary App ID -> Save.
- 4 Continue -> Register.
- 5 **Descargar el .p8** (solo se puede descargar una vez - guardar bien).
- 6 Anotar **Key ID** y **Team ID** (esquina superior derecha del dashboard).

11.4.4 Formatear la private key para .env

Los archivos `.env` no soportan multilinea. Hay que reemplazar los saltos de linea del `.p8` por `\n` literal (no un salto real).

```
# Extraer el PEM del .p8 (es un .p8 en realidad contiene un PEM dentro)
cat AuthKey_XXXXXXXXXX.p8 | base64 -d > key.pem

# El archivo suele verse asi:
# -----BEGIN PRIVATE KEY-----
# MIIEvAIBADANBgkqhkiG9w0BAQEFAAOCB7EwggSsMI...
# ...
# -----END PRIVATE KEY-----

# Escapar saltos de linea para que quepa en una sola linea:
PRIVATE_KEY=$(cat key.pem | tr '\n' '\\n')
echo "APPLE_PRIVATE_KEY=\"\$PRIVATE_KEY\""
```

11.4.5 Setear envs de Apple en cPanel

```
APPLE_CLIENT_ID=cl.cualautocompro.web.prod      # el Service ID del paso 11.4.2
APPLE_KEY_ID=ABC1234567                        # Key ID del paso 11.4.3
APPLE_TEAM_ID=XYZ9876543                       # Team ID del dashboard
APPLE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\nMIIIE...\n-----END PRIVATE KEY-----"
```

Importante: el valor de `APPLE_PRIVATE_KEY` debe tener comillas dobles en `.env` porque contiene espacios; los saltos de linea son `\n` literal, no saltos reales.

NO commitear este valor. Si el `.p8` se filtra, hay que revocarlo desde Apple Developer inmediatamente.

11.5 Cookies, CORS y SameSite en produccion

El backend setea una cookie `httpOnly` llamada `auth` con configuracion automatica segun el entorno. En produccion:

```
Set-Cookie: auth=<jwt>; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=604800
```

- HttpOnly:** la cookie no es accesible desde JavaScript (mitiga XSS).
- Secure:** solo se envia sobre HTTPS (obligatorio para OAuth en prod).
- SameSite=Lax:** protege contra CSRF. Enviar al backend solo en navegaciones top-level y en requests al mismo sitio.
- Max-Age=7d:** 7 dias. Refresca con cada login OAuth exitoso.
- Domain automatico:** el backend NO setea Domain en prod. El navegador usa el dominio que la emite (tipicamente el mismo que sirve el frontend).

Si backend y frontend estan en subdominios distintos (por ejemplo `api.cualautocompro.cl` y `app.cualautocompro.cl`) tendra que editar `apps/backend/src/modules/auth/auth-cookie.ts` y agregar `domain: '.cualautocompro.cl'` (con punto inicial para incluir subdominios). En este mismo cPanel donde ambos quedan en el mismo host, no es necesario.

11.6 Verificacion post-deploy del OAuth

Despues del primer deploy con las envs OAuth seteadas, verificar en orden:

- 1 Endpoint providers:** `curl https://cualautocompro.cl/api/v1/auth/providers` debe devolver `{"data":{"google":true,"apple":false},"error":null}`. Si devuelve `google:false`, falta `GOOGLE_CLIENT_ID` o `GOOGLE_CLIENT_SECRET`, o la consistencia fallo (envs parciales). Si devuelve 404, falta la migracion Prisma.
- 2 Healthcheck del backend:** `curl https://cualautocompro.cl/health` debe devolver `{"status":"ok",...}`.
- 3 UI en navegador:** ir a `https://cualautocompro.cl/login`. Deben aparecer los botones OAuth en la parte inferior del formulario.
- 4 Flujo end-to-end:** click en el boton, autorizar, volver al sitio. Verificar: (a) que la cookie `auth` aparece en DevTools -> Application -> Cookies, (b) que el header del sitio muestra tu email o nombre.

11.7 Problemas frecuentes especificos de OAuth

Lista de sintomas especificos de OAuth que pueden aparecer al desplegar en cPanel. Complementa la seccion 9 ('Problemas frecuentes') con casos OAuth.

Pantalla en blanco tras autorizar en Google

Causa: Google redirige a `BACKEND_ORIGIN/api/v1/auth/google/callback` (puerto 3000 o el interno del servidor). El navegador hace GET a esa URL, el backend procesa y redirige a `WEB_ORIGIN/?oauth=ok`. Si `WEB_ORIGIN` no esta en `https://` o no coincide con el origen del frontend, el navegador rechaza la redireccion o sale de la sesion.

Solucion: Confirmar que `WEB_ORIGIN` sea exactamente `https://cualautocompro.cl` (sin slash final, sin `www`). Confirmar que el CDN/proxy de cPanel no redirija a otra cosa.

redirect_uri_mismatch en Google

Causa: La URL registrada en Google Cloud Console no coincide exactamente con el `redirect_uri` que el backend pasa al provider. Tipicos errores: tener `http` en vez de `https`, tener `www.`, haber registrado el puerto 3000 o el path `/oauth/callback` (debe ser `/api/v1/auth/google/callback`).

Solucion: Ir a Google Cloud Console -> Credentials -> OAuth client -> Authorized redirect URIs. Confirmar que diga `https://cualautocompro.cl/api/v1/auth/google/callback`. Una sola entrada. Borrar las incorrectas. Esperar 30 segundos (cambios tardan en propagar) y reintentar.

Google muestra pantalla 'App is being verified'

Causa: Si la app esta en modo Testing (tipico recién creada) y el email del usuario NO esta en Test users, Google bloquea con una pantalla de verificacion.

Solucion: Agregar el email a `OAuth consent screen` -> `Test users`. O publicar la app (que requiere revision de Google si usa scope sensible - el scope `openid email profile` usado aca es NO sensible asi que el proceso es mas liviano).

Cookie no aparece despues del callback

Causa: El browser rechazo aceptar la cookie. Causas: (1) HTTPS no esta activo en el server (cookie tiene `Secure` en prod), (2) el frontend este en otro subdominio sin `Domain=` configurado, (3) el browser esta en modo incognito que a veces bloquea cookies de third-party.

Solucion: Confirmar que `https://` sirve correctamente. Verificar en DevTools -> Network -> el callback retorna una cabecera `Set-Cookie: auth=...`. Si retorna pero el browser no la guarda, es problema de `Secure` o `SameSite`.

Login funciona pero el usuario no aparece logueado al volver

Causa: El callback setea la cookie, redirige al frontend, pero el frontend no la esta leyendo. Usualmente porque el frontend hace fetch a un origen distinto al que la cookie pertenece.

Solucion: Verificar que `fetch(... {credentials: 'include'})` se este usando (ApiService ya lo hace). Confirmar que la URL del fetch (vs la URL de la cookie) comparten scheme + host (el puerto es OK con `Domain=localhost` en dev, en prod depender del `Domain`).

Apple falla con `invalid_client` o `JWT signature invalid`

Causa: El `APPLE_PRIVATE_KEY` esta mal. Comunes: (1) el `.p8` todavia contiene las lineas envolventes `-----BEGIN PRIVATE KEY-----` con saltos reales (deben ser `\n` literales), (2) el header/footer fue truncado al pegar en el panel de cPanel.

Solucion: Re-generar el string con el comando del 11.4.4. Verificar que en el panel de cPanel el valor muestre saltos de linea literales (en algunos panels hay que escapar las comillas manualmente). Probar localmente primero con `APPLE_PRIVATE_KEY` en un `.env.local`.

`OAuth_EMAIL_NOT_VERIFIED` al login con Google

Causa: El usuario de Google tiene email no verificado (tipico cuentas corporativas con SSO o cuentas nuevas). Google **NO** autentica usuarios sin email verificado.

Solucion: Pedir al usuario que use otra cuenta, o verificar primero el email en `https://myaccount.google.com/email`. No hay workaround.

11.8 Como desactivar OAuth si algo falla

Si despues del deploy algo no anda y queremos volver al login email/password mientras se investiga:

- 1 En cPanel -> Setup Node.js App -> Environment variables, dejar vacias o borrar `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `APPLE_CLIENT_ID`, `APPLE_KEY_ID`, `APPLE_TEAM_ID`, `APPLE_PRIVATE_KEY`.
- 2 Reiniciar la app desde el panel (boton Restart).
- 3 `GET /auth/providers` ahora devuelve `{"google":false,"apple":false}` y los botones desaparecen del frontend. Login email/password sigue funcionando normalmente.
- 4 La migracion Prisma `oauth_identity` NO se revierte (no es necesario). El codigo del backend sigue compilando con el modulo auth disponible.
- 5 Para volver a activar: rellenar las envs y reiniciar.

Esto es seguro: el codigo de login email/password es independiente de OAuth. No hay race condition ni estado intermedio que pueda romper sesiones existentes.

Fin de la guía. Si tiene dudas durante el despliegue, consulte primero la seccion 'Problemas frecuentes' (incluye problemas OAuth en 11.7) y luego a su proveedor de hosting.